

Introduction

This package ships image classification and object detection demos based on the Amlogic NN SDK. Models, sample images, and prebuilt Yocto executables (32-bit and 64-bit) are included so you can validate the NN Delegate quickly.

Quick Start (32-bit board example)

1. Copy the required shared libraries to the target

```
adb push
nnsdk_v2.8.5_2025_0801_merged/lib/linux/lib32_yocto/libAm1TFDelegate.so
/usr/lib
adb push nnsdk_v2.8.5_2025_0801_merged/lib/linux/lib32_yocto/libnnsdk.so
/usr/lib
```

2. Pick the demo you want to run

- Classification: use the assets under `case/classify/`.
- Detection: use `case/detect/` (an annotated `*_det.jpg` will be created after each run).

3. Copy demo files to the board

```
adb push case/classify /tmp/classify
adb push case/detect /tmp/detect
```

4. Run on the board

- Classification:

```
adb shell
cd /tmp/classify
chmod +x tf_delegate_classify_32
./tf_delegate_classify_32 mobilenet_v2_float32.tflite fish_224x224.jpeg
0
```

The console prints top-5 predictions with confidence scores.

```
./tf_delegate_classify_32 mobilenet_v2_float32.tflite fish_224x224.jpeg
0
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Use CPU backend.

input tensors: 1
output tensors: 1
input[0], shape=[1,224,224,3], zp=0, scale=0.000000, type=0
output[0], shape=[1,1000,-1,-1], zp=0, scale=0.000000, type=0
iw = 224, ih = 224, n = 3, input_size = 150528
outdata->out[0].size = 4000
top 0:score--18.842995,class--1
top 1:score--10.817957,class--29
top 2:score--9.467067,class--0
top 3:score--9.397186,class--120
```

```
top 4:score--9.059180,class--794
```

- o Detection:

```
adb shell
cd /tmp/detect
chmod +x tf_delegate_detect_32
./tf_delegate_detect_32 yolov8n_uint8.tflite zidane.jpg 0
```

Detection results are printed and the annotated `zidane_det.jpg` is stored alongside the input image.

```
sh-3.2# ./tf_delegate_detect_32 yolov8m_416-416_lout_int8.tflite
zidane.jpg 0
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Use CPU backend.

input tensors: 1
output tensors: 3
input[0], shape=[1,416,416,3], zp=0, scale=0.003922, type=2
output[0], shape=[1,26,26,144], zp=198, scale=0.263085, type=2
output[1], shape=[1,52,52,144], zp=177, scale=0.224821, type=2
output[2], shape=[1,13,13,144], zp=197, scale=0.225071, type=2
iw = 1280, ih = 720, n = 3, resized=[416,416,3], input_size = 519168
outdata->out[0].size = 97344
outdata->out[1].size = 389376
outdata->out[2].size = 24336
Detections (count=4):
  class=person (id=0) score=0.937 box=[129.2, 201.2, 1115.3, 715.5]
  class=person (id=0) score=0.922 box=[746.4, 40.7, 1136.2, 709.9]
  class=tie (id=27) score=0.663 box=[433.8, 436.3, 521.4, 720.0]
  class=tie (id=27) score=0.565 box=[984.0, 304.2, 1127.4, 705.7]
Annotated image saved to zidane_det.jpg
```



5. GPU mode (optional)

The last command argument selects the backend: `0` for CPU, `1` for GPU.

Directory Layout

- `build.sh`: Rebuild script for Yocto 32/64 binaries.
- `case/`: Ready-to-use executables, models, and sample images for the demos.
- `example/`: Source code and CMake configs for classification and detection.
- `nnsdk_v2.8.5_2025_0801_merged/`: Amlogic NN SDK headers and libraries.
- `3rdparty/`: stb image utilities.

Environment Setup & Rebuild (optional)

Prepare CMake

1. Visit [CMake Releases](#).
2. Download `cmake-3.24.0-linux-x86_64.tar.gz` or newer.
3. Extract to a local folder, e.g. `/opt/cmake-3.24.0-linux-x86_64`.
4. Update `build.sh`:

```
CMAKE_BIN="${CMAKE_BIN:-/opt/cmake-3.24.0-linux-x86_64/bin/cmake}"
```

Install Yocto toolchains

1. Obtain the SDK installer scripts from Amlogic:
 - 32-bit: `poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh`
 - 64-bit: `poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh`
2. Place the script under your target directory (e.g. `/opt/yocto-toolchain`) and run it:

```
./poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh
```

3. After installation, configure `build.sh`:

```
YOCTO_SDK_ROOT_32="${YOCTO_SDK_ROOT_32:-/opt/yocto-toolchain/32}"
YOCTO_SDK_ROOT_64="${YOCTO_SDK_ROOT_64:-/opt/yocto-toolchain/64}"
```

Rebuild

```
./build.sh classify    # Produces case/classify/tf_delegate_classify_32/64
./build.sh detect     # Produces case/detect/tf_delegate_detect_32/64
```

Notes

- GPU mode requires proper hardware support and drivers on the device.
- The detection demo saves annotated images as JPG beside the input file.
- To test other models or inputs, copy them into the appropriate `case/` subdirectory and adjust the command parameters.

